Universidad Mariano Gálvez de Guatemala

Curso: Desarrollo Web

GUÍA DEL PROYECTO

Sistema de Punto de Venta e Inventario «Las Casas del Rincón Guatemalteco»

Explicación sencilla de qué se construyó, cómo se diseñó y cómo funciona por dentro: requerimientos, modelo C4, casos de uso, DDD, arquitectura hexagonal, base de datos, diseño de interfaz, pruebas, Kanban e instalación.

Estudiante	Andrés Ixcajoc López
Carné	090-23-19115
Curso	Desarrollo Web
Catedrático	Jorge Mario Moya Marroquín

Guatemala, octubre de 2026

Contenido

1. ¿Qué problema resolvemos? 3
2. Lo que pidió el enunciado y cómo se cumplió 4
3. Cómo se construyó: el proceso paso a paso 6
4. Tecnologías usadas y por qué 7
5. Modelo C4: el sistema visto con zoom 8
6. Casos de uso: qué puede hacer cada actor 13
7. DDD: diseñar a partir del negocio 17
8. Arquitectura hexagonal: puertos y adaptadores 21
9. MVC y estructura del proyecto 27
10. Base de datos 28
11. El sistema funcionando: una venta completa 29
12. Diseño de la interfaz (UI/UX) 35
13. Pruebas y organización (Kanban) 36
14. Cómo instalarlo y presentarlo en otra computadora 38
15. Glosario para estudiantes 39
16. Conclusiones y enlaces 40

1. ¿Qué problema resolvemos?

«Las Casas del Rincón Guatemalteco» es un negocio que vende productos de consumo diario a las personas que viven en colonias residenciales. Necesitaba un sistema para **vender** (como la caja de un supermercado) y para **controlar su inventario**: saber qué mercadería entra, qué se vende, cuánto dinero ingresa y cuánto sale.

Los productos se agrupan en **7 familias**:

Familia	Ejemplos
Pastas	Fideos, corbatitas, coditos, spaghetti
Productos Cárnicos	Pollo y carne de cerdo
Embutidos	Jamones, longanizas, salchichas
Lácteos	Crema, queso, leche
Golosinas	Dulces, chocolates, galletas
Aguas Gaseosas	Bebidas gaseosas
Carbonatados	Gatorade, Revive, bebidas energizantes

¿Quiénes usan el sistema?

Actor	Qué hace
Cajero	Abre la caja, despacha los productos, cobra, emite la factura, atiende devoluciones y hace el corte de caja.
Administrador	Mantiene los catálogos, registra compras, configura promociones, administra usuarios y revisa reportes.
Cliente	Compra, recibe su factura con un código QR y descubre su promoción con el celular.
Core de Consumo Masivo	Sistema que ya existía en el negocio y del que se obtienen productos, proveedores y precios.

> **Idea clave**
>
> En una frase: es un **punto de venta (POS) web** que además lleva el **inventario**, la **caja**, las **promociones con QR** y los **reportes** del negocio.

2. Lo que pidió el enunciado y cómo se cumplió

Cada requerimiento del documento del proyecto tiene una pantalla en el sistema. Esta tabla sirve como lista de verificación durante la presentación:

#	Requerimiento	Cómo lo resuelve el sistema
1	Interfaz de acceso (todo a nivel de base de datos)	Login: usuario y contraseña cifrada validados contra la tabla `usuarios`; roles y permisos también en la base.
2	Catálogos: clientes, proveedores, productos y precios	Listados con búsqueda y fichas; precios normal y de mayoreo editables en línea.
3	Despacho: lectura, cobro y subtotal por línea, total de la compra	Se escanea el código de barras; cada lectura suma a su línea y se recalculan subtotales, descuentos y total.
4	Devoluciones por daño o vencimiento	Se busca la factura, se eligen las unidades y el motivo; el producto sale como baja y el dinero sale de la caja.
5	Cobro con tarjeta de crédito, débito o efectivo	Efectivo con teclado numérico y cálculo de cambio; tarjeta con número de autorización del voucher.
6	Facturación con datos del cliente y todos los productos	NIT (o CF), nombre, dirección, detalle, descuentos, total; en pantalla, impresa y en PDF.
7	Reportes: cierre del día, devoluciones, clientes, compras	Cuatro reportes con rango de fechas, en pantalla y PDF.
8	Código QR en la factura que muestra la promoción	El QR abre una página pública en el celular con la promoción ganada, su vigencia y su código.
9	Menú interactivo con las opciones del software	Barra superior por módulos; cada rol ve solo lo que tiene permitido.
10	Apertura de caja (monto inicial) y corte de caja (cierre)	Apertura con monto inicial; corte con efectivo esperado, contado y diferencia.
10a	Promociones: producto gratis, vale, 2.º a mitad, mayoreo, puntos	Los cinco tipos, automáticos en el despacho o entregados por el QR de la factura.
—	Control del inventario que entra y que se vende	Compras a proveedores, existencias y kardex (historial de cada producto).
—	Integración con el Core de Consumo Masivo (existente)	Sincronización de productos, proveedores y precios mediante un puerto y un adaptador.

Especificaciones técnicas

Pedido	Solución
Software libre y ambiente web	PHP 8.3 con el framework Laravel 13; páginas web con Blade y Bootstrap 5.
Base de datos libre y relacional	PostgreSQL 16 (ejecutado en Docker).
Modelo de desarrollo 3 capas o MVC	MVC de Laravel, integrado con la arquitectura hexagonal (sección 9).
Modelo C4	Cuatro diagramas: contexto, contenedores, componentes y código (sección 5).
DDD y arquitectura hexagonal	10 contextos del negocio; capas Dominio, Aplicación e Infraestructura (secciones 7 y 8).
Core de consumo masivo existente	Puerto `CatalogoCorePort` con dos adaptadores: archivo o API HTTP.
Modelo Kanban	Tablero en GitHub Projects (sección 13).

3. Cómo se construyó: el proceso paso a paso

El proyecto no empezó escribiendo código. Se siguió el orden que usa un equipo de software real: entender, diseñar, construir, probar y mejorar.

1. **Analizar el enunciado**
   Se leyeron los requerimientos y se identificaron actores, procesos (vender, cobrar, devolver, comprar) y reglas del negocio.

2. **Diseñar la arquitectura**
   Se dibujó el modelo C4 (de lo general a lo detallado) y los diagramas de casos de uso de cada actor.

3. **Modelar el dominio (DDD)**
   Se dividió el negocio en 10 contextos (Caja, Despacho, Promociones...) con su propio vocabulario y reglas.

4. **Definir puertos y adaptadores**
   Para cada contexto se dibujó su hexágono: qué casos de uso ofrece y qué necesita del exterior.

5. **Programar**
   Primero el dominio en PHP puro, luego los casos de uso, después la base de datos, los controladores y las pantallas.

6. **Probar**
   Pruebas automáticas del dominio, del flujo completo de una venta y de todas las pantallas.

7. **Organizar con Kanban**
   Cada tarea fue una tarjeta que avanzó de «Por hacer» a «En progreso» y «Terminado».

8. **Mejorar la interfaz (UI/UX)**
   Se revisó el diseño, se hizo un mockup en Figma, se aprobó y se aplicó al código.

> **Idea clave**
> Diseñar antes de programar evita rehacer trabajo: cuando llegó el momento de programar, ya se sabía qué módulos existían, cómo se comunicaban y qué reglas debían cumplir.

4. Tecnologías usadas y por qué

Tecnología	Qué es	Por qué se eligió
PHP 8.3	Lenguaje de programación del lado del servidor.	Libre, muy usado en la web y requerido como ambiente web.
Laravel 13	Framework (conjunto de herramientas) para hacer aplicaciones web en PHP.	Trae rutas, validaciones, sesiones, migraciones y pruebas listas; sigue el patrón MVC.
PostgreSQL 16	Base de datos relacional.	Libre, robusta y con transacciones: una venta se guarda completa o no se guarda.
Docker	Ejecuta programas en «contenedores» aislados.	Levanta PostgreSQL con un comando, igual en cualquier computadora.
Blade + Bootstrap 5	Plantillas HTML de Laravel y librería de estilos.	Pantallas rápidas de construir y que se adaptan al tamaño de la pantalla.
DomPDF	Librería que convierte HTML en PDF.	Factura y reportes en PDF.
Simple QrCode	Librería que genera códigos QR.	El QR de promoción de cada factura.
PHPUnit	Herramienta de pruebas automáticas.	Verificar que las reglas del negocio funcionen siempre.
Git, GitHub y GitHub Projects	Control de versiones y tablero Kanban.	Historial del código y seguimiento de tareas.
Figma	Herramienta de diseño de interfaces.	Mockup de todas las pantallas antes de cambiar el código.

5. Modelo C4: el sistema visto con zoom

El modelo C4 describe un sistema como si usáramos un mapa con zoom. Cada nivel acerca la vista un poco más:

Nivel	Analogía	Qué muestra en este proyecto
1 · Contexto	Ver el país en el mapa	El sistema POS, quiénes lo usan (cajero, administrador, cliente) y el Core de Consumo Masivo.
2 · Contenedores	Ver la ciudad	Las piezas que se ejecutan: interfaz web, vista móvil del QR, aplicación Laravel y base PostgreSQL.
3 · Componentes	Ver los barrios	Los módulos dentro de Laravel (Caja, Despacho, Promociones…) y sus adaptadores.
4 · Código	Ver las casas	Las clases del módulo de despacho: controlador, caso de uso, entidad Venta, repositorio.

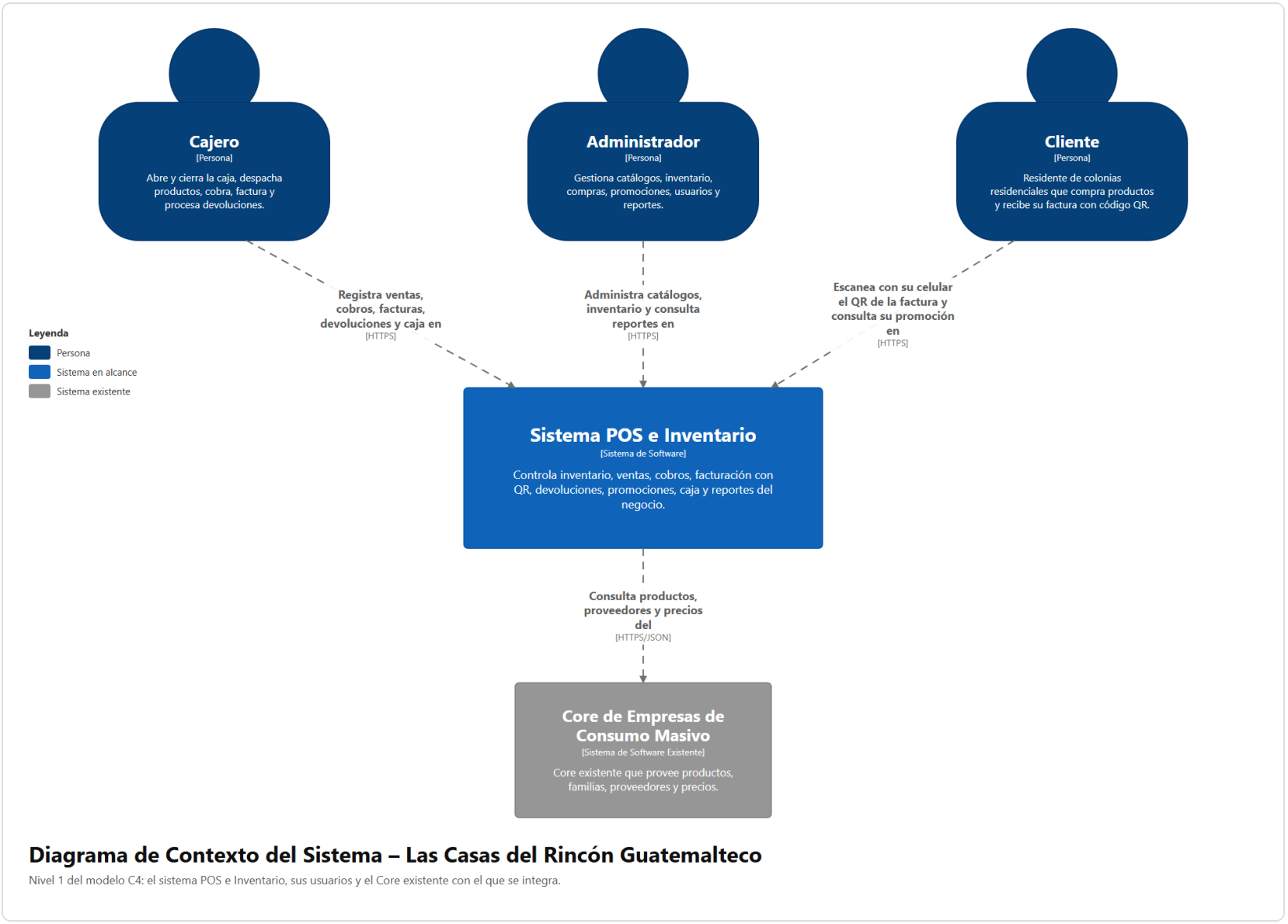


Figura 1. Nivel 1 – Contexto.

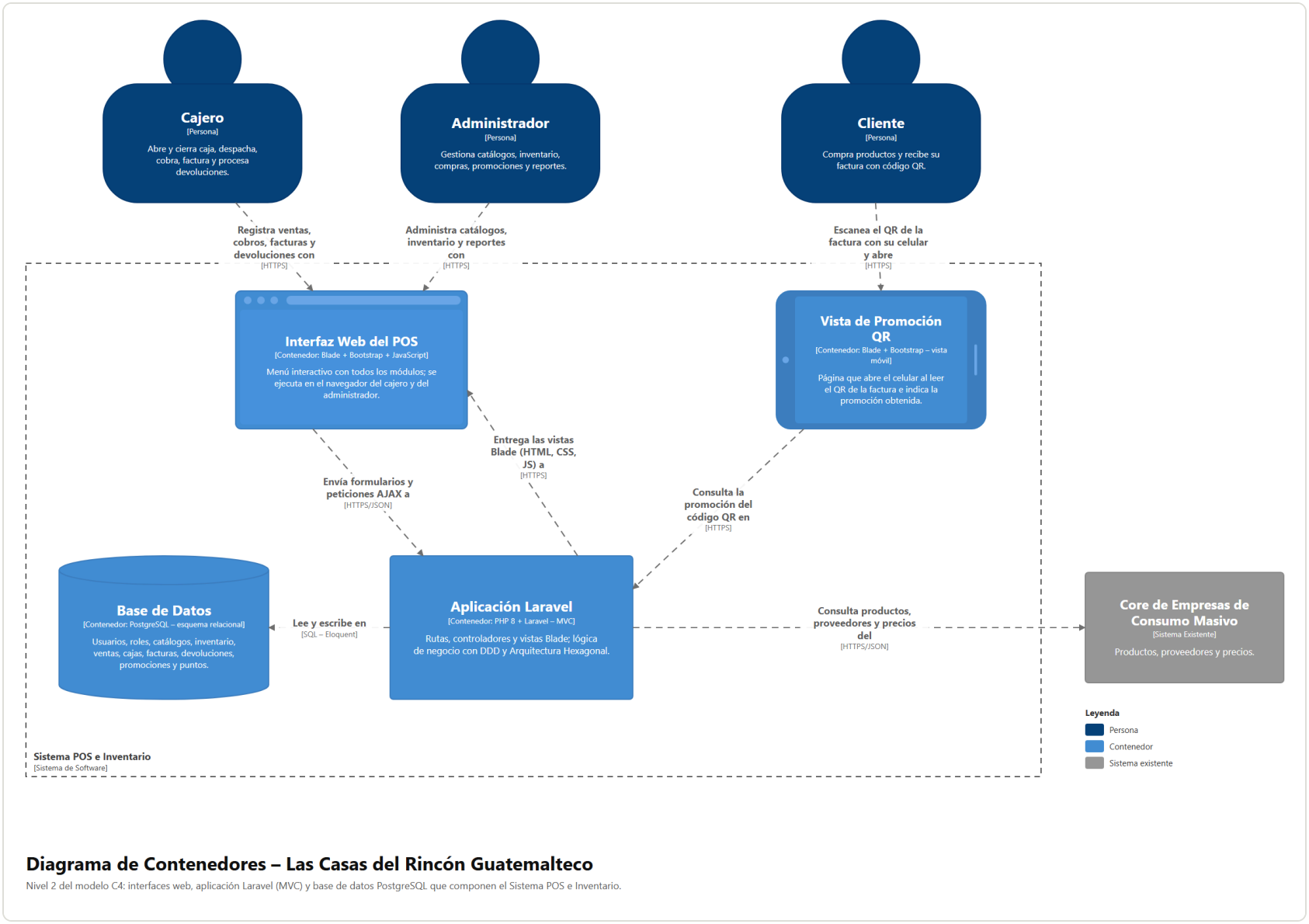


Diagrama de Contenedores – Las Casas del Rincón Guatemalteco

Nivel 2 del modelo C4: interfaces web, aplicación Laravel (MVC) y base de datos PostgreSQL que componen el Sistema POS e Inventario.

Figura 2. Nivel 2 – Contenedores.

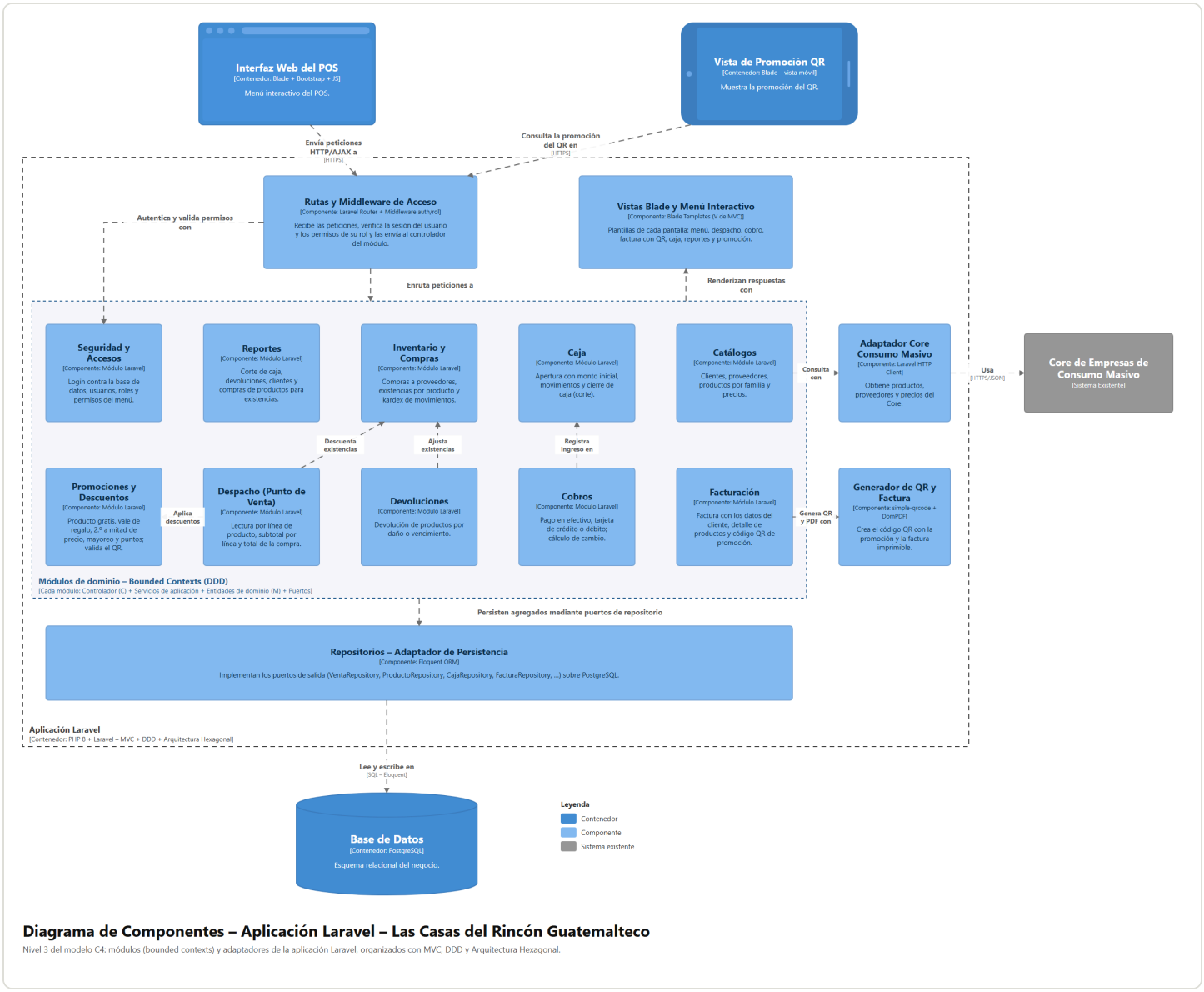
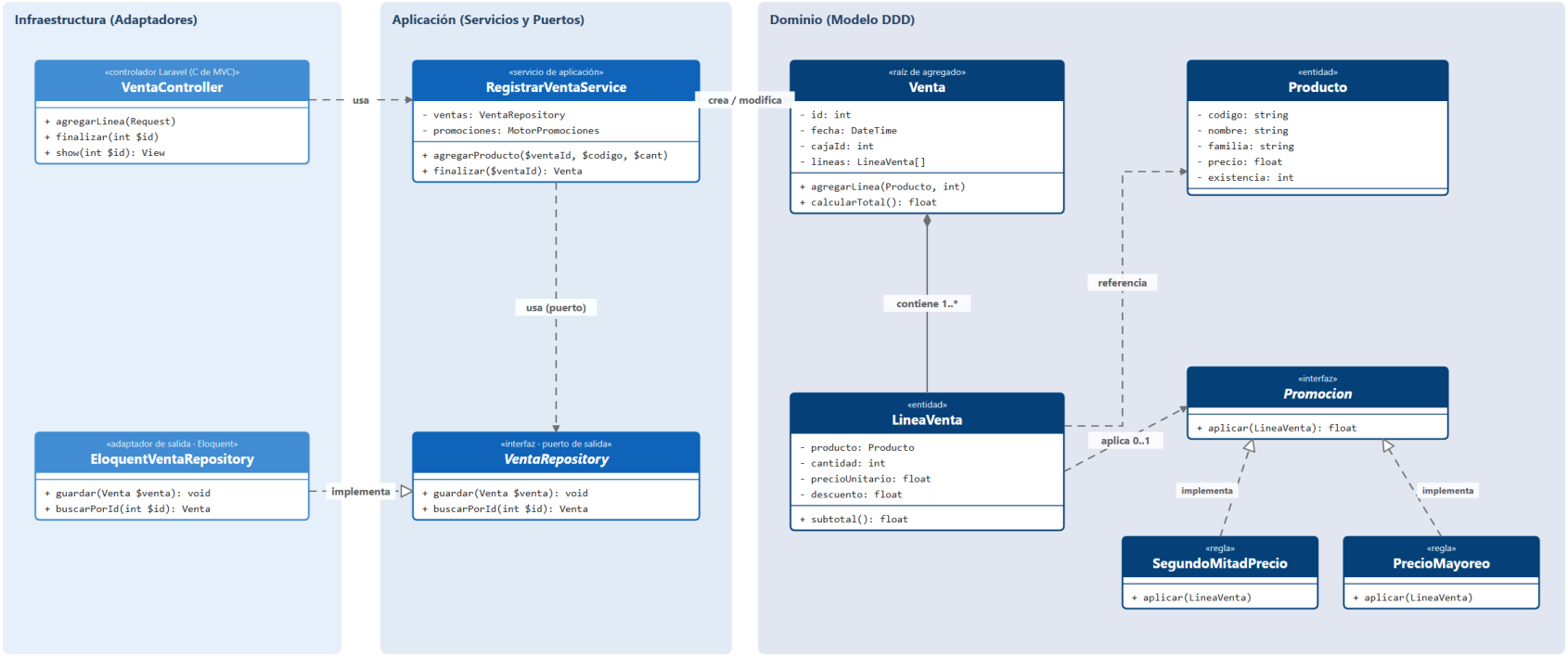


Diagrama de Componentes – Aplicación Laravel – Las Casas del Rincón Guatemalteco

Nivel 3 del modelo C4: módulos (bounded contexts) y adaptadores de la aplicación Laravel, organizados con MVC, DDD y Arquitectura Hexagonal.

Figura 3. Nivel 3 – Componentes de la aplicación Laravel.




```php
// app/Domain/Venta/Venta.php  (fragmento ilustrativo)
class Venta
{
    private array $lineas = [];

    public function agregarLinea(Producto $producto, int $cantidad): void
    {
        if ($producto->existencia < $cantidad) throw new StockInsuficiente();
        $this->lineas[] = new LineaVenta($producto, $cantidad);
    }

    public function calcularTotal(): float
    {
        return array_sum(array_map(fn($l) => $l->subtotal(), $this->lineas));
    }
}
```

Relaciones: ——▷ usa / dependencia - - -▷ (triángulo) implementa ◆—— composición
Flujo: el controlador recibe la lectura del producto → el servicio carga la Venta, aplica la promoción, recalcula subtotales/total y la guarda a través del puerto VentaRepository.

Diagrama de Código – Componente Despacho (Punto de Venta)

Nivel 4 del modelo C4: clases principales del módulo de despacho en Laravel con Arquitectura Hexagonal (representación simplificada).

Figura 4. Nivel 4 – Código del componente de despacho.

6. Casos de uso: qué puede hacer cada actor

Un caso de uso es **una acción que un actor realiza con el sistema** para lograr algo, por ejemplo «Cobrar la venta» o «Leer el QR de la factura». Los diagramas muestran a los actores (monigotes) y sus acciones (óvalos):

- **«include»**: un paso que siempre ocurre (cobrar *incluye* registrar el ingreso en caja).
- **«extend»**: un paso opcional (en el despacho se *puede* canjear un cupón).
- **Generalización** (flecha con triángulo): variantes de una misma acción (los tres métodos de pago o los cinco tipos de promoción).

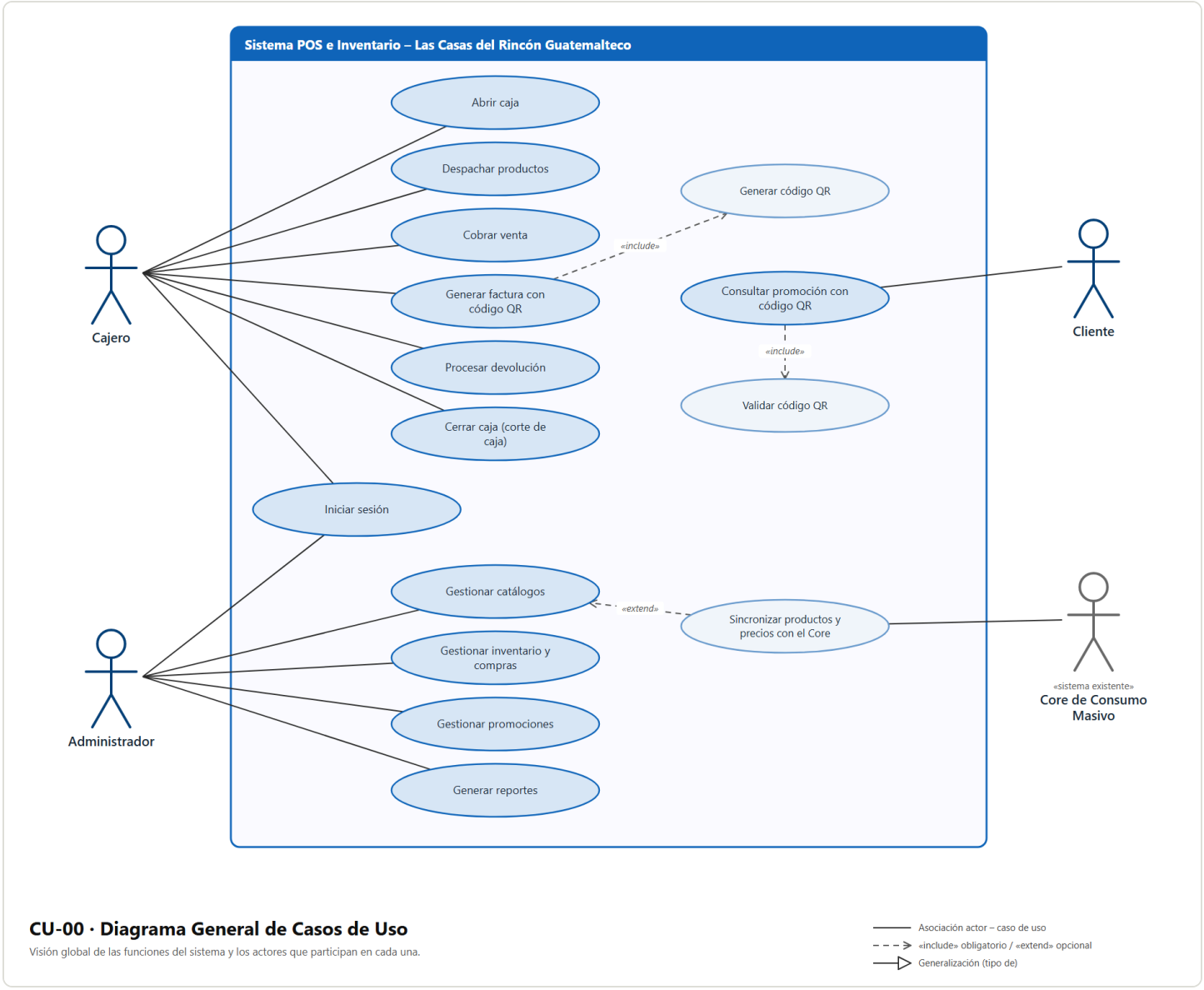


Figura 5. Diagrama general de casos de uso.

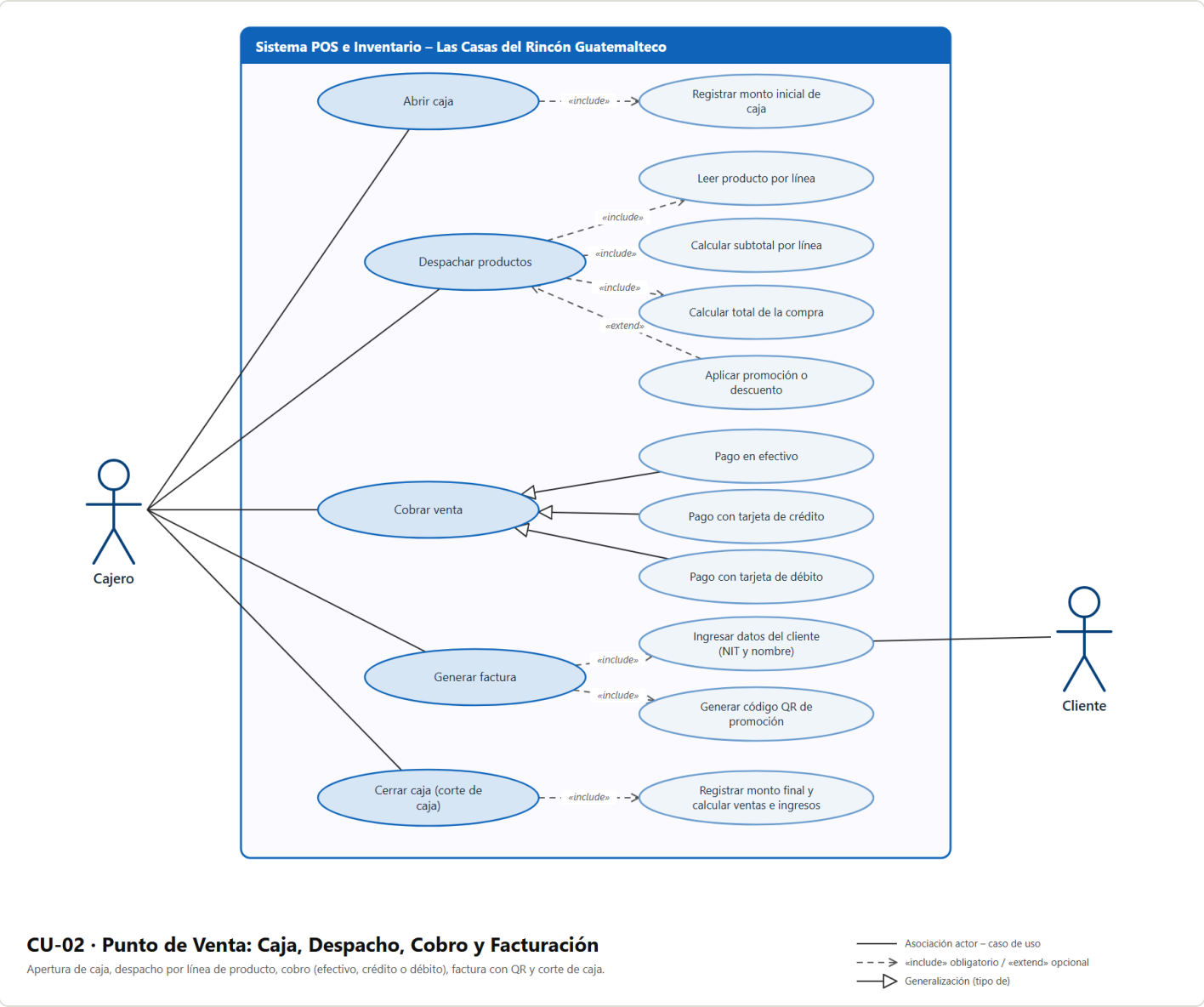


Figura 6. Casos de uso del punto de venta.

Los demás diagramas (acceso y catálogos, devoluciones e inventario, promociones y QR, reportes) están en `docs/casos-de-uso` del repositorio.

7. DDD: diseñar a partir del negocio

Domain-Driven Design (DDD), o diseño guiado por el dominio, significa que el software se organiza **igual que el negocio**. El «dominio» es el negocio mismo: ventas, caja, promociones, inventario. En lugar de empezar por tablas o pantallas, se empieza preguntando: *¿qué hace el negocio y cuáles son sus reglas?*

7.1 Subdominios: no todo es igual de importante

Tipo	Contextos	Por qué
Principal	Despacho, Promociones, Cobros, Facturación	Es donde el negocio gana dinero y se diferencia (la venta con promociones y QR).
Soporte	Catálogos, Inventario y Compras, Caja, Devoluciones	Necesarios para operar, propios del negocio.
Genérico	Seguridad y Accesos, Reportes	Problemas que cualquier sistema tiene: iniciar sesión, permisos, consultas.

7.2 Contextos delimitados: el negocio dividido en 10 partes

Un **contexto delimitado** es una parte del negocio con su propio vocabulario y sus propias reglas. Es como los departamentos de una tienda: cada uno sabe hacer lo suyo.

Contexto	Responsabilidad	Conceptos principales
Seguridad y Accesos	Iniciar sesión, roles y permisos del menú	Usuario, Rol, Permiso
Catálogos	Clientes, proveedores, productos y precios; conexión con el Core	Cliente, Proveedor, Producto
Inventario y Compras	Mercadería que entra, existencias y kardex	Compra, MovimientoInventario
Caja	Apertura, ingresos, egresos y corte	Caja, MovimientoCaja
Despacho (Ventas)	Leer productos, subtotales y total	Venta, LineaVenta
Promociones	Descuentos, cupones QR y puntos	Promocion, Cupon, MotorPromociones
Cobros	Efectivo, tarjeta de crédito o débito	Pago, MetodoPago
Facturación	Factura y cupón QR	Factura

Contexto	Responsabilidad	Conceptos principales
Devoluciones	Devolución por daño o vencimiento	Devolucion, MotivoDevolucion
Reportes	Consultas de solo lectura	—

7.3 Lenguaje ubicuo: el código habla como el negocio

Los nombres de las clases y métodos son las mismas palabras del enunciado. Por eso el código se entiende leyéndolo:

En el negocio se dice...	En el código existe...
«Abrir la caja con un monto inicial»	`AbrirCajaUseCase::abrir($cajero, $montoInicial)`
«Agregar un producto a la venta»	`Venta::agregarProducto(...)`
«Canjear el cupón del QR»	`GestionarVentaUseCase::canjearCupon($venta, $token)`
«Hacer el corte de caja»	`Caja::cerrar($ingresos, $egresos, $montoContado, $ahora)` calcula esperado y diferencia
«Devolver por daño o vencimiento»	`MotivoDevolucion::Dano`, `MotivoDevolucion::Vencimiento`

7.4 Piezas de DDD usadas

Pieza	Qué es (en simple)	Ejemplo en el sistema
Entidad	Algo con identidad propia que cambia con el tiempo.	Producto, Cliente, Usuario
Agregado	Un grupo de objetos que se guarda y valida junto; tiene un «jefe» (raíz).	**Venta** con sus **LineaVenta**: nadie modifica una línea sin pasar por la venta.
Objeto de valor	Un valor sin identidad, definido por lo que vale.	MetodoPago, TipoPromocion, Dinero
Servicio de dominio	Una regla que involucra a varios objetos.	**MotorPromociones** aplica las promociones a una venta.
Repositorio	Una «bodega» para guardar y buscar agregados, sin decir cómo.	VentaRepository, ProductoRepository
Capa anticorrupción	Traductor para que un sistema externo no «contamine» nuestro modelo.	CatalogoCoreNormalizador traduce el formato del Core.

7.5 Reglas del negocio que el dominio protege

- La existencia de un producto nunca queda negativa; cada entrada o salida queda en el kardex.
- No se despacha ni se cobra sin una caja abierta; cada cajero tiene una sola caja abierta.
- Subtotal de línea = cantidad × precio − descuento; total = suma de subtotales − vale − puntos.
- En efectivo el monto recibido debe cubrir el total; con tarjeta se exige la referencia del voucher.
- Solo se factura una venta pagada; sin NIT se factura a CF (consumidor final).
- Un cupón QR se usa una sola vez y antes de su vencimiento.
- Solo se devuelve hasta la cantidad vendida y con motivo (daño o vencimiento).

7.6 Ejemplo real: la regla de existencias vive en la Venta

```
// app/Domain/Ventas/Venta.php  (PHP puro: no usa Laravel ni la base de datos)
public function agregarProducto(int $productoId, ..., int $cantidad, int $existencia): void
{
    $this→asegurarEnProceso();
    $nueva = $this→cantidadDe($productoId) + $cantidad;      // lectura por línea: suma a
la línea existente
    if ($nueva > $existencia) {
        throw new ReglaNegocioException("Existencia insuficiente de {$nombre}: disponible
{$existencia}.");
    }
    ...
}
```

7.7 Mapa de contextos

Los contextos se ayudan entre sí a través de **puertos** (interfaces). Por ejemplo, Cobros no escribe directamente en la tabla de la caja: le pide a Caja, por medio de `CajaPort`, que registre el ingreso.

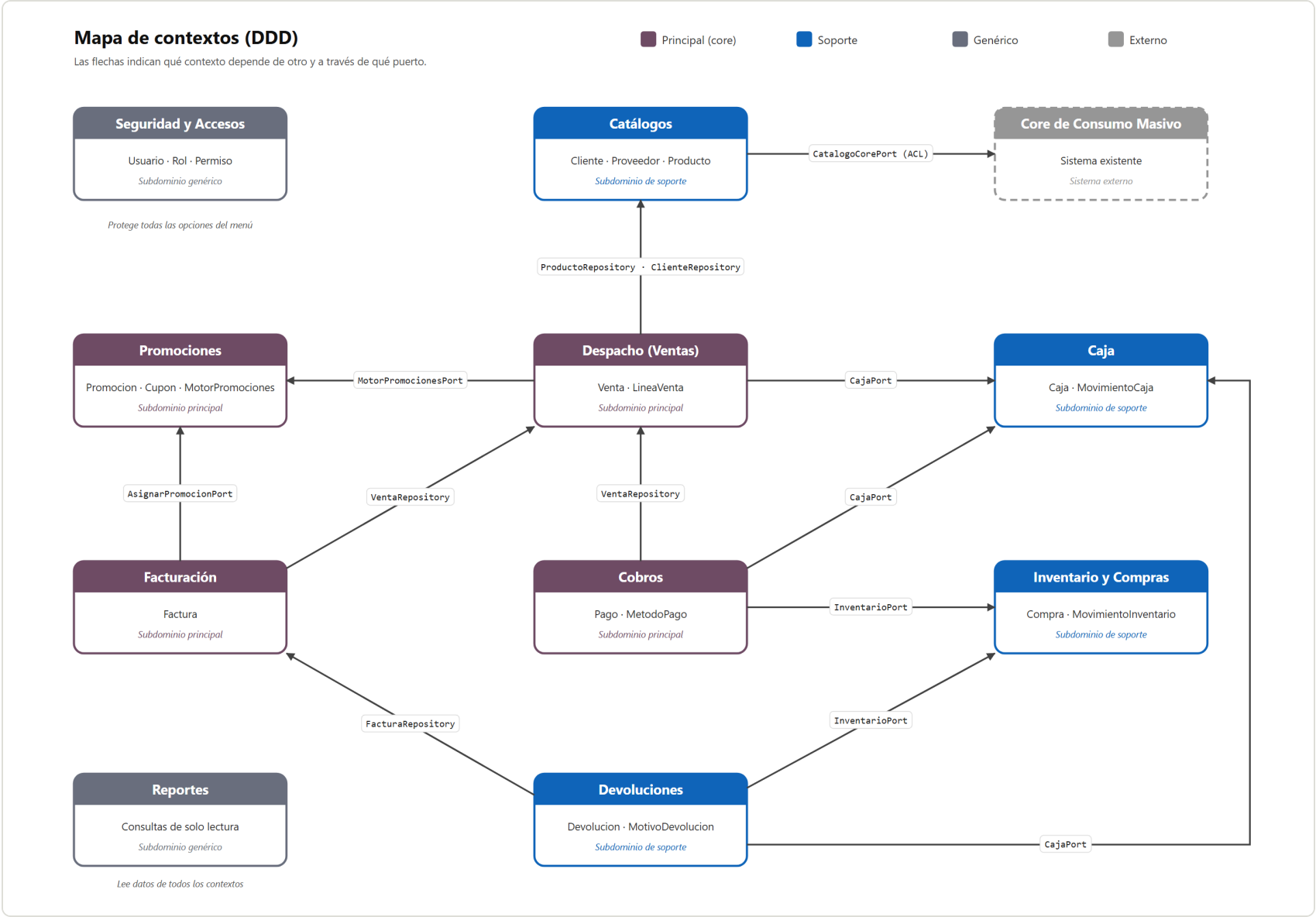


Figura 7. Mapa de contextos: quién depende de quién y por qué puerto.

8. Arquitectura hexagonal: puertos y adaptadores

La arquitectura hexagonal protege el corazón del sistema (el dominio) de los detalles técnicos. La mejor analogía es un **enchufe**: el aparato (dominio) define la forma del enchufe (**puerto**) y cada país usa un **adaptador** distinto. Si cambia el tomacorriente, se cambia el adaptador, no el aparato.

Capa	Carpeta	Qué contiene	¿Conoce Laravel?
Dominio (centro)	`app/Domain`	Entidades y reglas del negocio: Venta, Caja, Promocion, MotorPromociones.	No
Aplicación (puertos)	`app/Application`	Casos de uso (*Ports/In*), lo que necesitan del exterior (*Ports/Out*) y su implementación (*Services*).	No
Infraestructura (adaptadores)	`app/Infrastructure`	Controladores web, repositorios con PostgreSQL, QR, PDF, conexión con el Core.	Sí

> **Idea clave**
> La regla de oro: **las dependencias apuntan hacia adentro**. Infraestructura → Aplicación → Dominio. El dominio nunca sabe si los datos vienen de PostgreSQL, de un archivo o de una prueba.

8.1 Recorrido real: el cajero escanea un producto

1. **Adaptador primario** — `DespachoController::agregar()` recibe el código desde la pantalla y valida el formulario.
2. **Puerto de entrada** — llama a la interfaz `AgregarProductoUseCase` (no sabe qué clase la implementa).
3. **Caso de uso** — `DespachoService` busca el producto con `ProductoRepository` (puerto de salida).
4. **Dominio** — `Venta::agregarProducto()` aplica las reglas; `MotorPromociones` calcula los descuentos.
5. **Adaptador secundario** — `EloquentVentaRepository` guarda la venta en PostgreSQL.

```
// 1. Puerto de entrada (app/Application/Ventas/Ports/In)
interface AgregarProductoUseCase {
    public function agregar(int $ventaId, string $codigo, int $cantidad): Venta;
}

// 2. Adaptador primario
(app/Infrastructure/Http/Controllers/Ventas/DespachoController.php)
public function agregar(Request $request, int $venta, AgregarProductoUseCase $agregar)
{
    $datos = $request→validate(['codigo' ⇒ 'required|string|max:40', 'cantidad' ⇒
'nullable|integer|min:1|max:999']);
    return $this→respuesta($request, $agregar→agregar($venta, $datos['codigo'], (int)
($datos['cantidad'] ?? 1)));
}

// 3. Ensamblaje (app/Providers/HexagonalServiceProvider.php): cada puerto con su adaptador
VentaRepository::class ⇒ EloquentVentaRepository::class,
InventarioPort::class  ⇒ InventarioAdapter::class,
```

8.2 La prueba de que funciona: el Core de Consumo Masivo

El puerto `CatalogoCorePort` tiene **dos adaptadores**: uno lee un archivo exportado por el Core y otro consume su API HTTP. Se cambia de uno a otro solo con una línea del archivo `.env` (`CORE_DRIVER=archivo` o `http`), **sin tocar el dominio**.



Figura 8. Vista general de la arquitectura hexagonal.

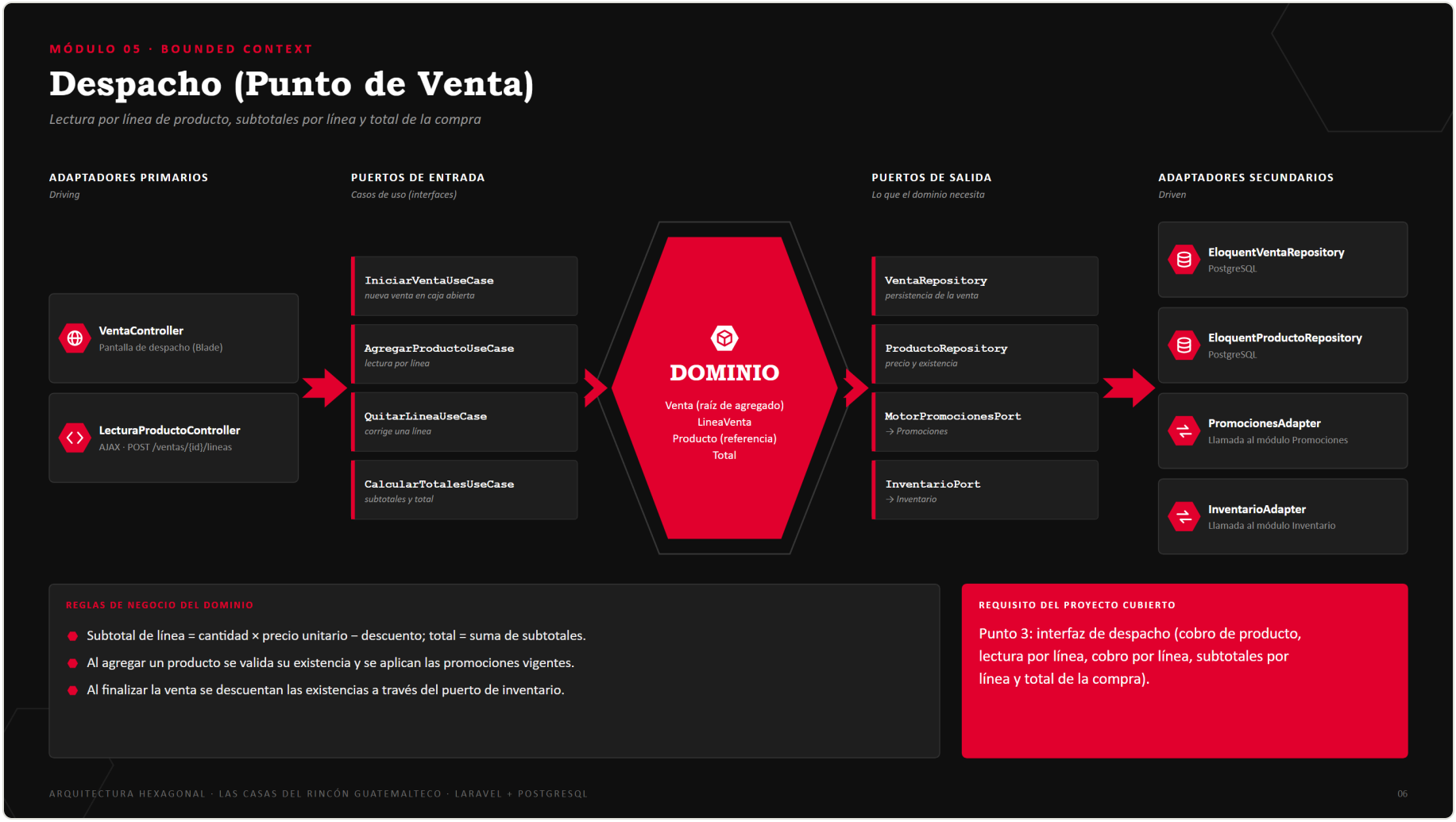


Figura 9. Hexágono del módulo de despacho.



Figura 10. Flujo de una venta entre contextos.

Hay un diagrama hexagonal por cada uno de los 10 módulos en `docs/arquitectura-hexagonal`.

9. MVC y estructura del proyecto

El enunciado pide el modelo **MVC** (Modelo–Vista–Controlador). Laravel ya lo usa, y encaja con la arquitectura hexagonal:

MVC	Dónde está	Ejemplo
Modelo	Reglas en `app/Domain` + persistencia en `Infrastructure/Persistence`	Venta (reglas) y VentaModel (tabla ventas)
Vista	Plantillas Blade en `resources/views`	despacho.blade.php, cobro.blade.php, factura.blade.php
Controlador	`app/Infrastructure/Http/Controllers`	DespachoController, CobroController, CajaController

```
sistema-pos/
├── app/
│   ├── Domain/            ← reglas del negocio (PHP puro), un folder por contexto
│   ├── Application/       ← casos de uso: Ports/In, Ports/Out y Services
│   ├── Infrastructure/    ← Http (controladores), Persistence (PostgreSQL), Qr, Pdf,
External (Core)
│   └── Providers/         ← HexagonalServiceProvider: conecta puertos con adaptadores
├── database/migrations/   ← creación de tablas      database/seeders/ ← datos iniciales y
de demostración
├── resources/views/       ← pantallas Blade         public/css/pos.css ← estilos «Jade &
Maíz»
├── routes/web.php         ← direcciones del sistema, protegidas por permiso
├── tests/                 ← pruebas automáticas
├── docs/                  ← diagramas, documentos y guías
└── scripts/               ← instalar.ps1 e iniciar.ps1
```

Dato	Cantidad
Clases PHP en app/	174
Clases del dominio puro	20
Contextos (módulos)	10
Rutas web	≈ 60
Pruebas automáticas	17 (147 verificaciones)

10. Base de datos

Las tablas se crean con **migraciones** de Laravel (archivos que describen cada tabla), así cualquier computadora obtiene exactamente la misma base con `php artisan migrate`. Todo, incluso el inicio de sesión, se valida contra la base de datos, como pide el enunciado.

Tabla(s)	Guarda	Se relaciona con
usuarios, roles, permisos, rol_permiso	Quién entra y qué módulos ve	usuario → rol; rol ↔ permisos
clientes · proveedores · familias	Catálogos básicos	—
productos	Código de barras, precios, existencia	→ familia, → proveedor
compras, detalle_compras	Mercadería que entra	→ proveedor; detalle → producto
movimientos_inventario	Kardex: entradas, salidas y bajas	→ producto
cajas, movimientos_caja	Apertura, ingresos, egresos y corte	→ usuario; movimiento → caja
ventas, detalle_ventas	Venta con subtotales, descuentos y total	→ caja, → cliente; detalle → producto
pagos · facturas	Cómo se pagó y la factura emitida	→ venta (1 a 1)
promociones · cupones	Reglas de descuento y códigos QR	cupón → promoción, → factura
devoluciones, detalle_devoluciones	Motivo y reembolso	→ factura, → caja

Idea clave

Las operaciones importantes usan **transacciones**: al cobrar se descuenta el inventario, se registra el pago y el ingreso en caja *todo junto*. Si algo falla, no se guarda nada a medias.

11. El sistema funcionando: una venta completa

Este es el recorrido que hace el cajero, que conecta casi todos los módulos:

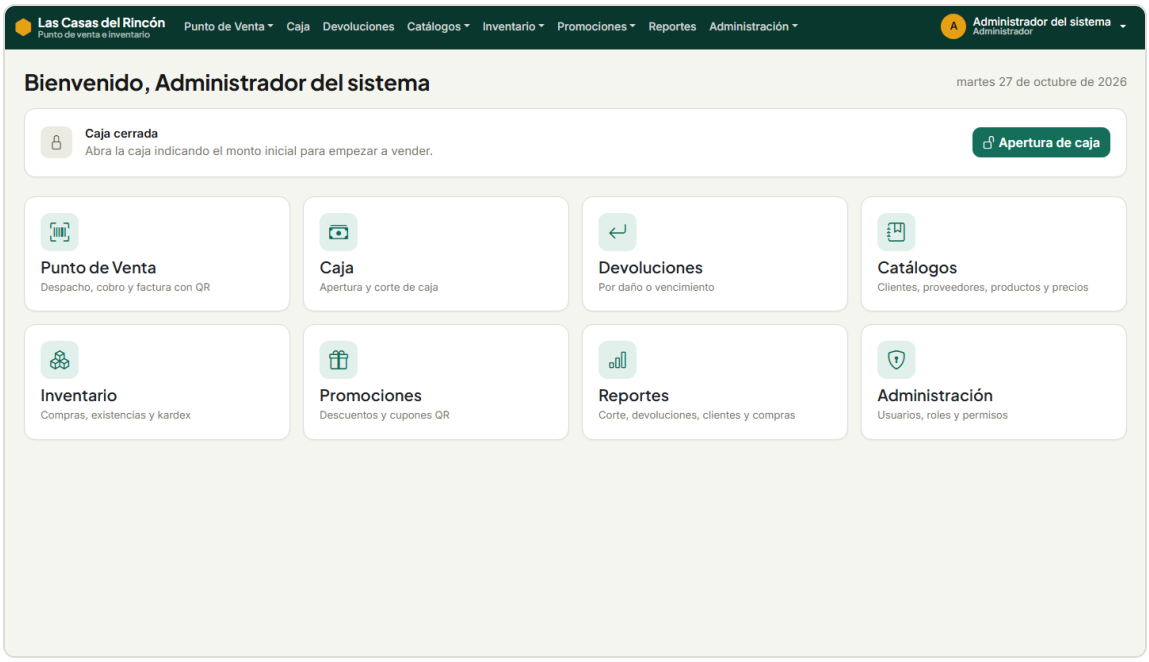


Figura 11. Inicio: menú interactivo por módulos (requerimiento 9).

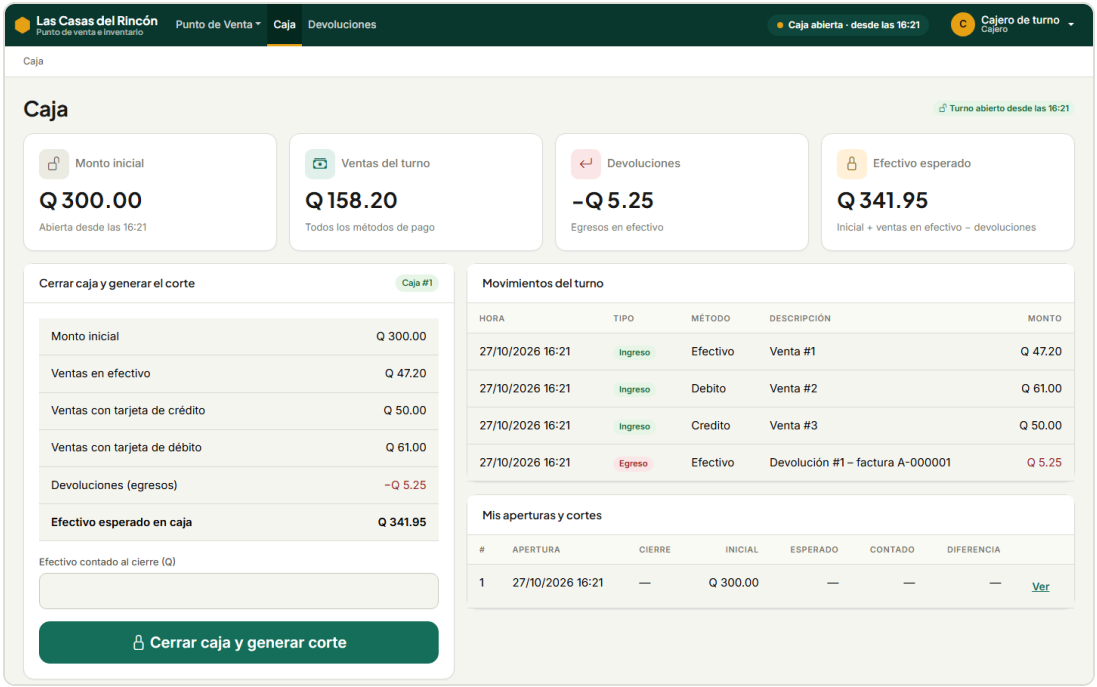


Figura 12. Caja abierta con indicadores del turno (requerimiento 10).

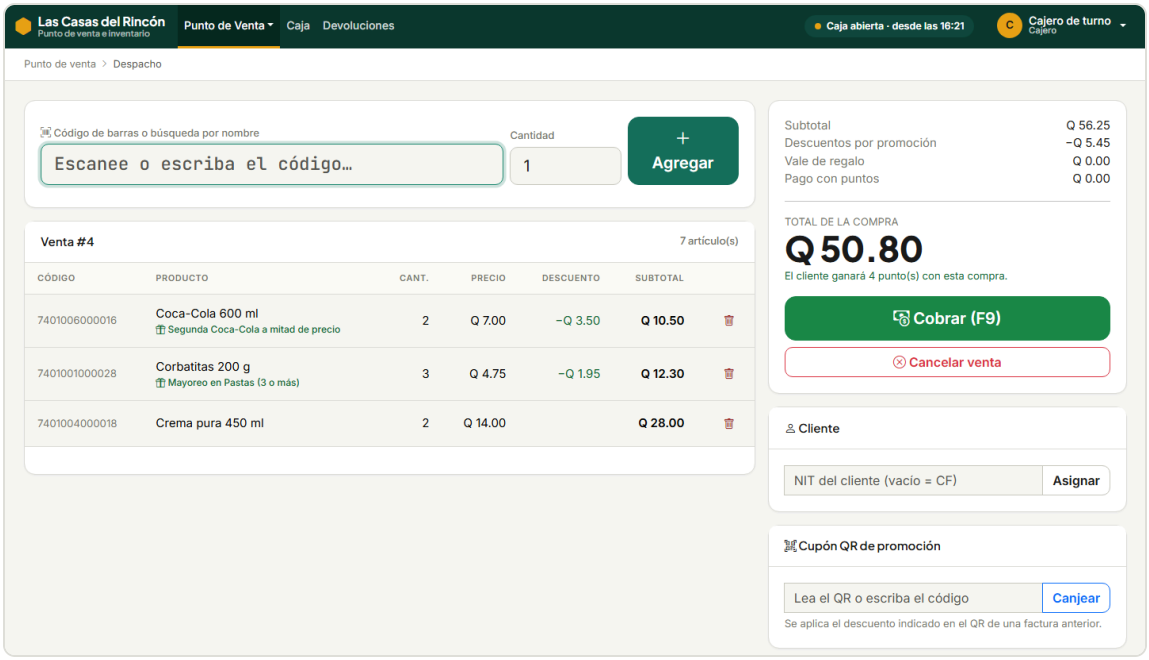


Figura 13. Despacho: lectura por línea, promociones y total (requerimiento 3).

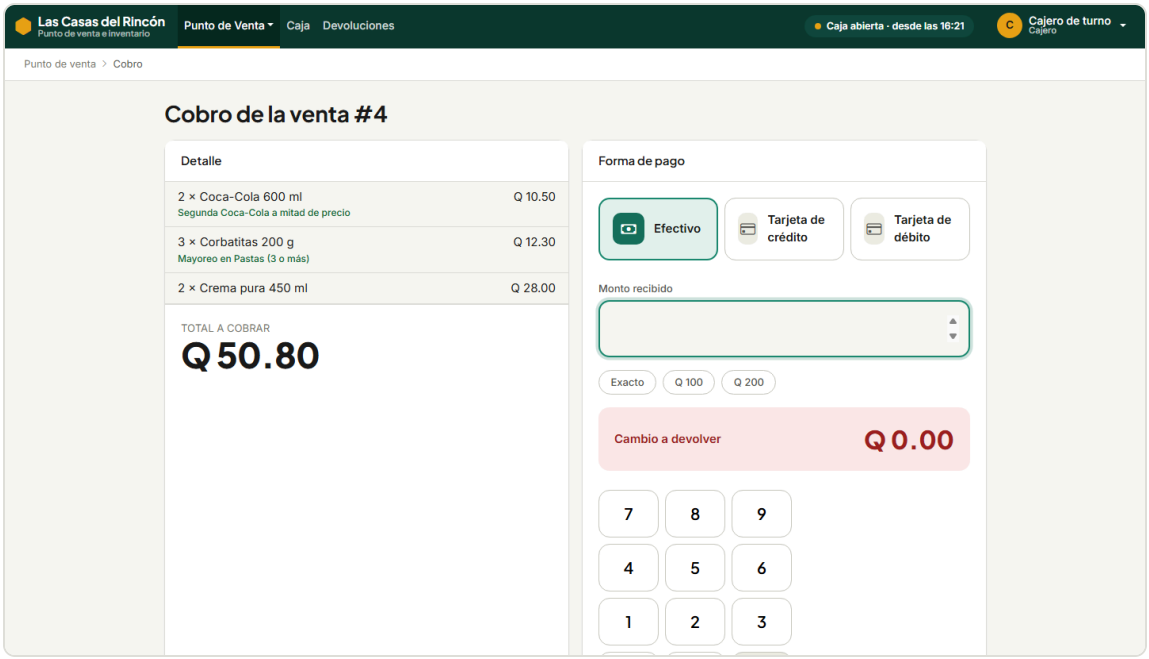


Figura 14. Cobro con teclado numérico, montos rápidos y cambio (requerimiento 5).

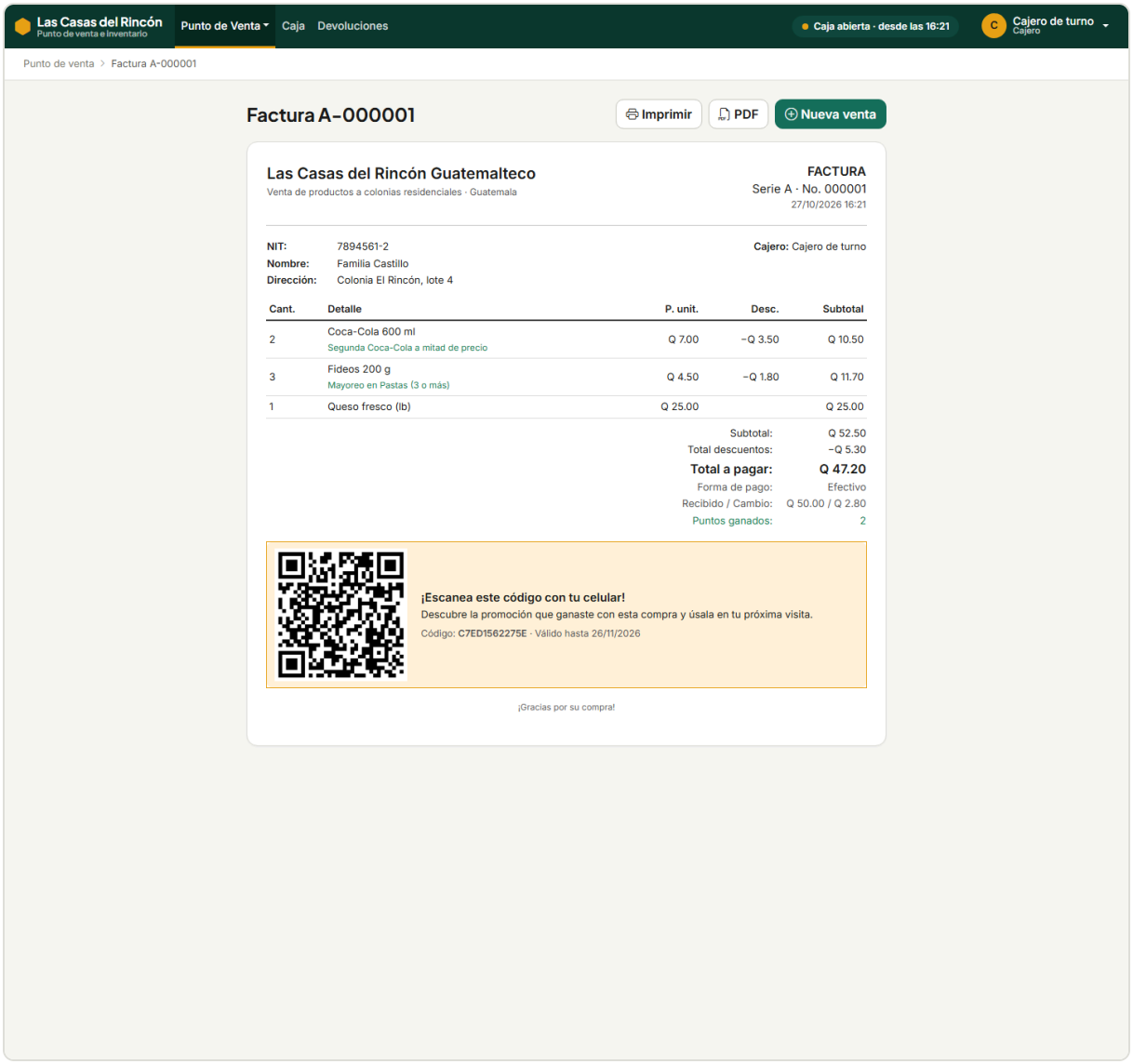


Figura 15. Factura con datos del cliente y código QR (requerimientos 6 y 8).

¿Cómo funciona el QR?

1. Al facturar, el sistema elige al azar una promoción vigente de tipo «QR» y crea un **cupón** con un código único y fecha de vencimiento.
2. El QR contiene una dirección web: `http://servidor/promocion/CÓDIGO`.
3. El cliente lo escanea con la cámara de su celular y ve una página pública con su promoción (no necesita iniciar sesión).
4. En su próxima compra el cajero escribe o escanea el código y el descuento se aplica. El cupón queda **canjeado** y no se puede volver a usar.



Figura 16. Lo que ve el cliente en el celular al leer el QR.

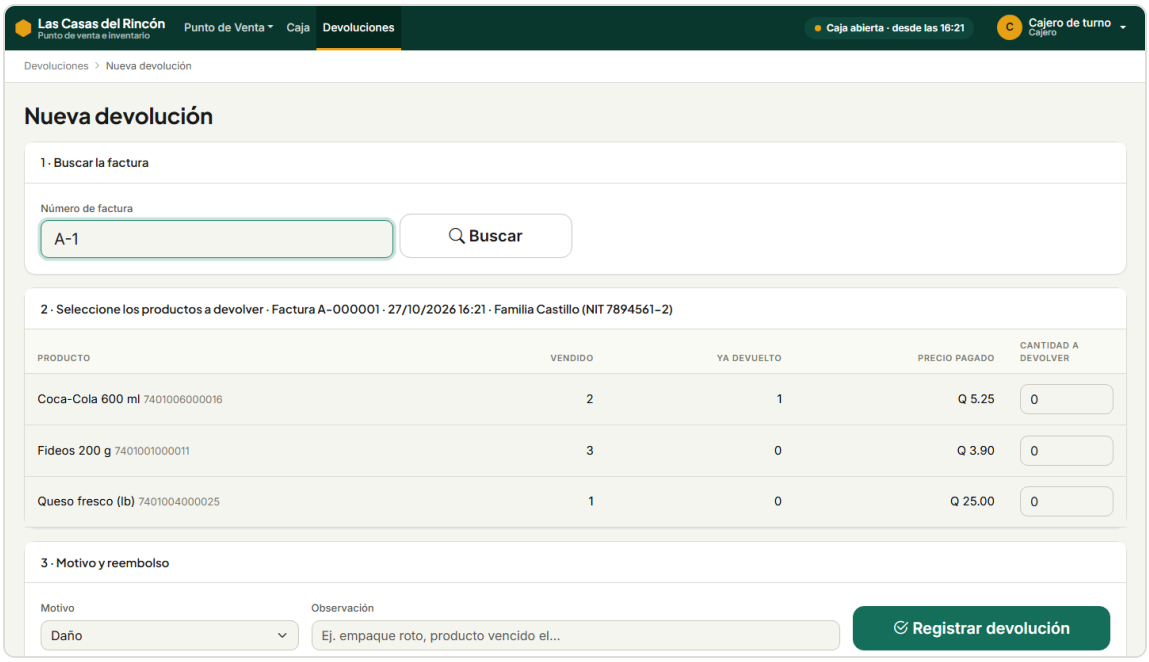


Figura 17. Devolución por daño o vencimiento en tres pasos (requerimiento 4).

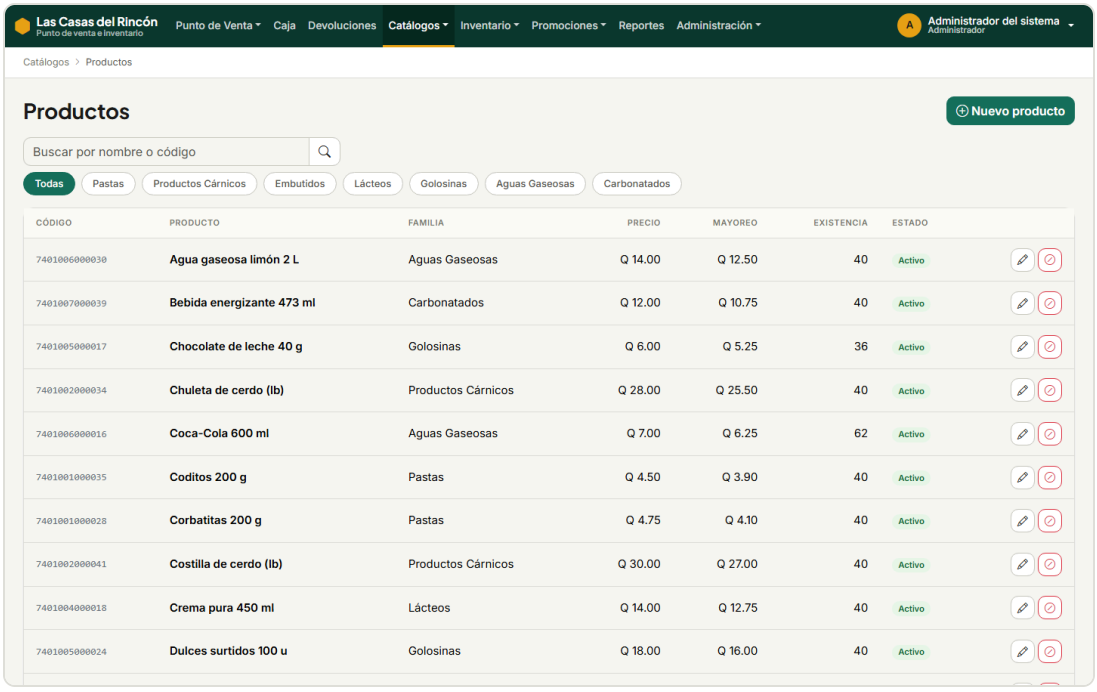


Figura 18. Catálogo de productos filtrado por familia (requerimiento 2).

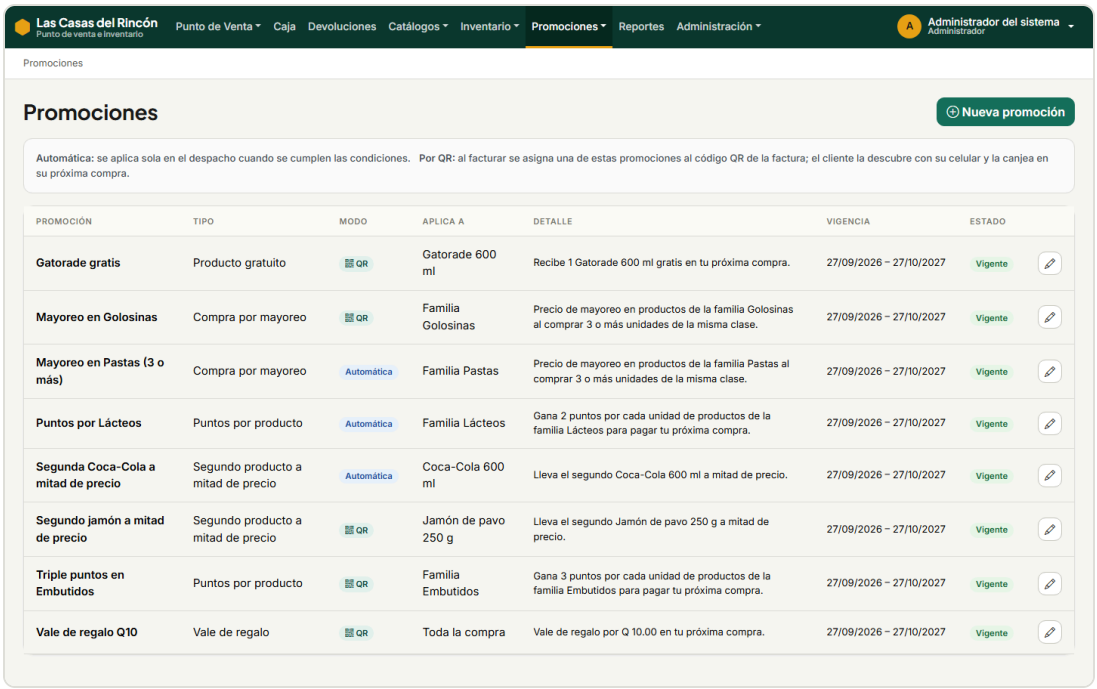


Figura 19. Los cinco tipos de promoción (requerimiento 10a).

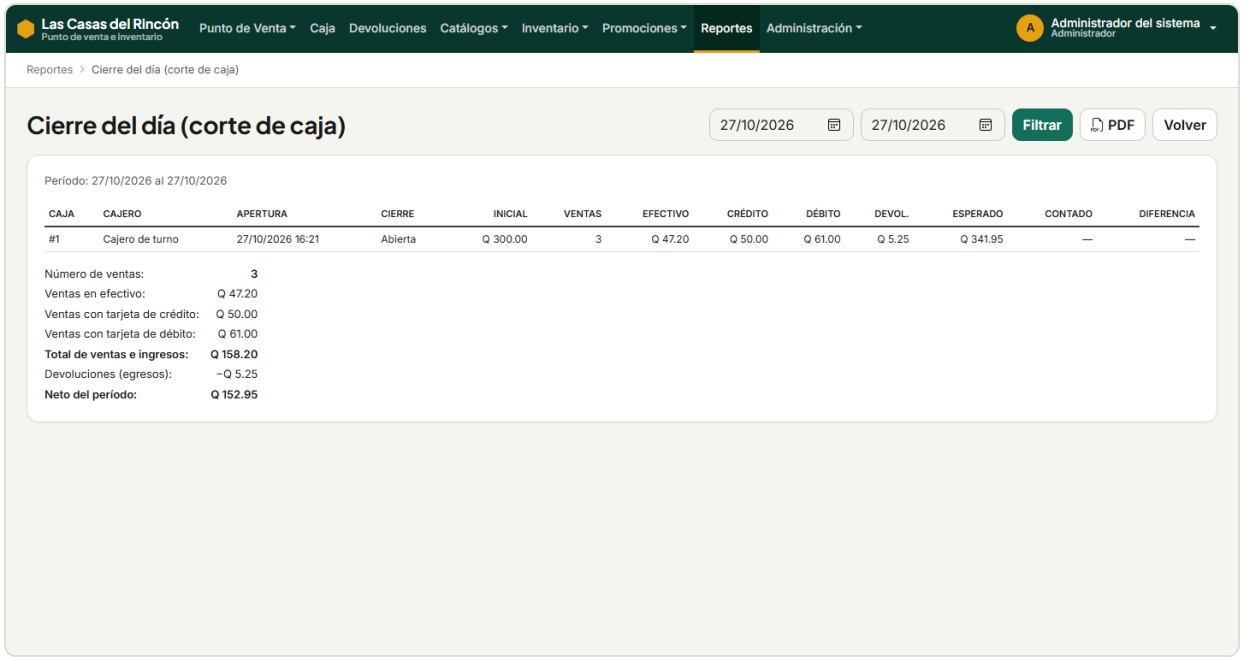


Figura 20. Reporte de cierre del día (requerimiento 7).

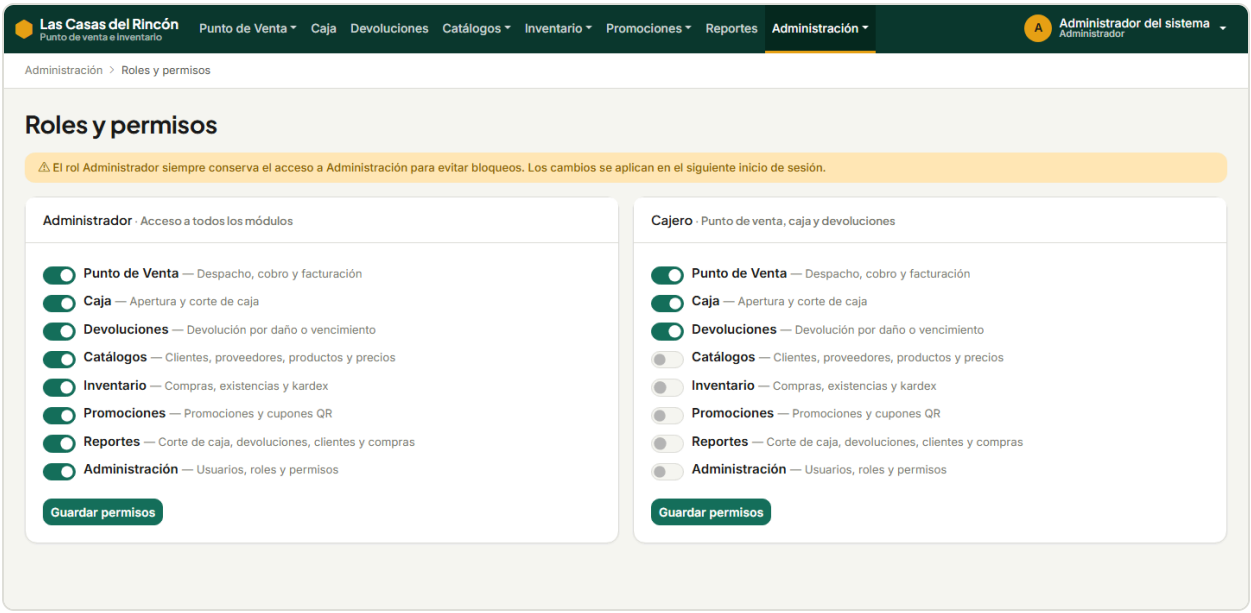


Figura 21. Roles y permisos: qué módulos ve cada rol.

12. Diseño de la interfaz (UI/UX)

Con el sistema funcionando, se hizo una **revisión de diseño**. La primera versión cumplía, pero usaba los mismos colores de Odoo (un ERP conocido) y se veía como una plantilla genérica. El proceso fue:

1. **Revisión**: se anotaron los problemas (sin teclado para cobrar, estado de la caja invisible, poco contraste…).
2. **Mockup en Figma**: se diseñaron más de 40 pantallas con un sistema de diseño propio (colores, tipografías, componentes). Enlace: `https://www.figma.com/design/X00YwF6wiM0zh9zMWQiNqq`
3. **Aprobación** del mockup y luego **aplicación al código** (CSS y vistas Blade), manteniendo las pruebas en verde.

Se mantuvo (estructura conocida)	Es propio (identidad)
Barra de módulos arriba, migas de pan, listas y fichas, búsqueda y filtros.	Paleta **jade** y **maíz**, tipografías Plus Jakarta Sans e Inter, componentes táctiles para el cobro, indicadores y estados con color.

Problema encontrado	Mejora aplicada
Cobro lento con pantalla táctil	Teclado numérico grande, montos rápidos y cambio destacado
No se veía si la caja estaba abierta	Indicador «Caja abierta» siempre visible en la barra
Sin orientación entre pantallas	Migas de pan y títulos consistentes
Estados difíciles de distinguir	Insignias de color (Disponible, Stock bajo, Agotado, Vigente…)
Inicio sin información	Indicadores del turno y accesos directos

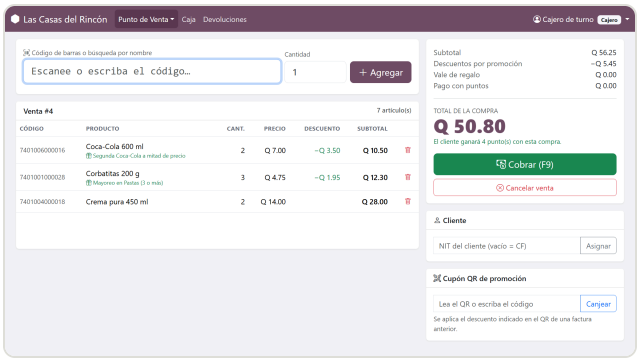


Figura 22. Antes: despacho con el diseño inicial.

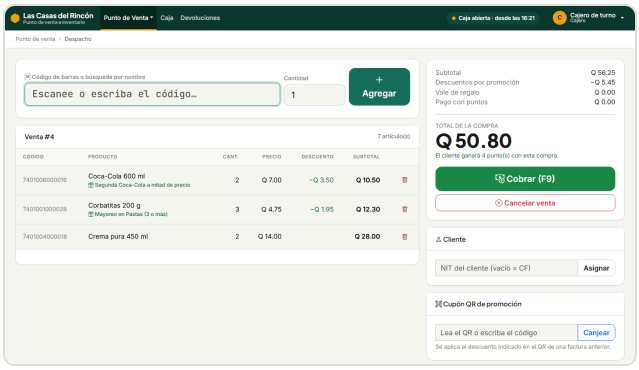


Figura 23. Después: despacho con la identidad «Jade & Maíz».

13. Pruebas y organización (Kanban)

13.1 Pruebas automáticas

Una prueba automática es un pequeño programa que usa el sistema y comprueba que el resultado sea el correcto. Se ejecutan todas con un comando y tardan unos segundos.

Grupo	Qué comprueba
Dominio (8 pruebas)	Subtotales, total, existencias, cada tipo de promoción, puntos, cobro y corte de caja — sin base de datos.
Flujo de venta (2 pruebas)	Caja → despacho → cobro → factura con QR → canje del cupón → devolución → corte.
Pantallas (7 pruebas)	Permisos por rol, todas las pantallas, la venta por la web, factura y reportes en PDF, Core.

Resultado: **17 pruebas y 147 verificaciones, todas correctas**. Gracias a ellas, el rediseño de la interfaz se hizo sin romper ninguna funcionalidad.

13.2 Kanban

Kanban es un tablero con columnas **Por hacer** → **En progreso** → **Terminado**. Cada tarea (diseño, módulo, pruebas, documentación) fue una tarjeta enlazada a un issue de GitHub. Así se veía en todo momento qué faltaba y qué estaba listo.

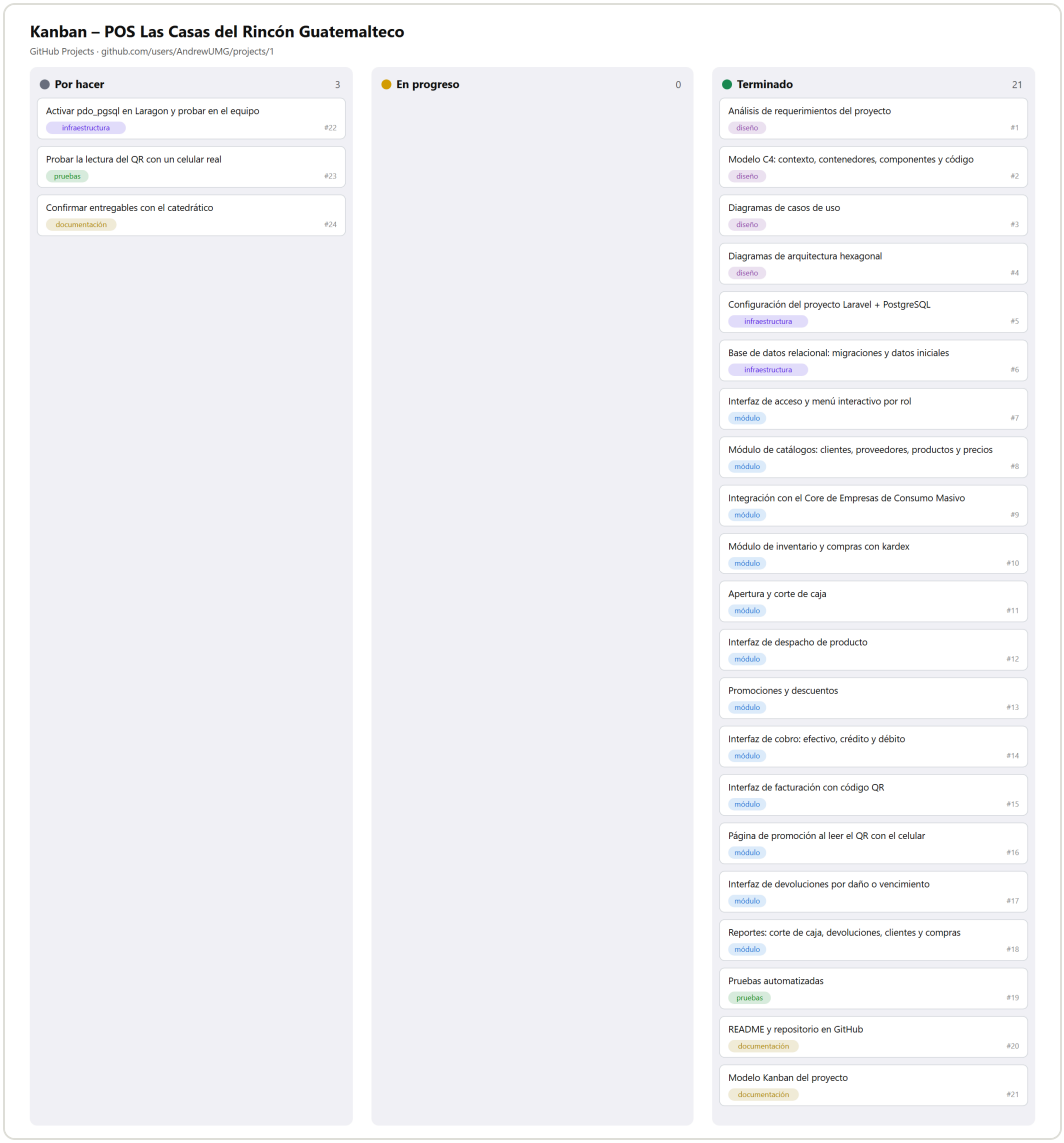


Figura 24. Tablero Kanban en GitHub Projects.

14. Cómo instalarlo y presentarlo en otra computadora

El repositorio incluye scripts para que cualquier persona lo monte en Windows con dos comandos.

Requisitos

- **Git** · **Laragon** (trae PHP 8.3 y Composer) · **Docker Desktop** encendido.

Pasos

```
git clone https://github.com/AndrewUMG/pos-las-casas-del-rincon.git
cd pos-las-casas-del-rincon
powershell -ExecutionPolicy Bypass -File scripts\instalar.ps1 -Demo   # instala todo y
carga datos de ejemplo
powershell -ExecutionPolicy Bypass -File scripts\iniciar.ps1          # abre
http://127.0.0.1:8000
powershell -ExecutionPolicy Bypass -File scripts\iniciar.ps1 -Red     # para leer el QR con
el celular (misma Wi-Fi)
```

Usuario	Contraseña	Rol
admin	admin123	Administrador (todos los módulos)
cajero	cajero123	Cajero (punto de venta, caja y devoluciones)

El script `instalar.ps1` verifica las herramientas, levanta PostgreSQL en Docker, instala las dependencias, crea las tablas y carga los datos. Con `-Demo` registra ventas, una devolución y una venta en proceso para presentar. Volver a ejecutarlo deja todo como nuevo.

Guía detallada y solución de problemas: `docs/INSTALACION.md`. Guion de la demostración en vivo: `docs/GUION_DEMO.md`.

15. Glosario para estudiantes

Término	Significado sencillo
POS	Point of Sale: la «caja registradora» digital donde se vende.
Framework	Conjunto de herramientas que resuelve lo común para que uno se concentre en el negocio (Laravel).
MVC	Separar los datos (Modelo), las pantallas (Vista) y quien coordina (Controlador).
C4	Cuatro diagramas de arquitectura, del más general al más detallado.
DDD	Diseñar el software con la forma y el vocabulario del negocio.
Contexto delimitado	Una parte del negocio con sus propias reglas (Caja, Promociones…).
Agregado	Grupo de objetos que se valida y guarda junto (Venta con sus líneas).
Puerto	Una interfaz: el «enchufe» que define qué se necesita.
Adaptador	La clase que cumple ese enchufe con una tecnología concreta (PostgreSQL, DomPDF…).
Caso de uso	Una acción del usuario con el sistema (cobrar, devolver, abrir caja).
Migración	Archivo que crea o cambia una tabla de la base de datos.
Seeder	Archivo que llena la base con datos iniciales o de ejemplo.
Kardex	Historial de entradas y salidas de un producto.
Docker	Programa que ejecuta otros programas en contenedores iguales en cualquier PC.
Mockup	Diseño visual de las pantallas antes de programarlas.

16. Conclusiones y enlaces

- Se cumplieron todos los requerimientos del enunciado, cada uno con su pantalla y verificado con pruebas automáticas.
- DDD permitió que el código hable el idioma del negocio y que las reglas importantes estén en un solo lugar.
- La arquitectura hexagonal separó el negocio de la tecnología: el Core funciona con archivo o API cambiando una línea.
- Diseñar primero (C4, casos de uso, hexágonos) y organizar con Kanban hizo el desarrollo ordenado y predecible.
- La revisión UI/UX con mockup en Figma mejoró la experiencia del cajero sin romper ninguna funcionalidad.

Recurso	Dirección
Código fuente	https://github.com/AndrewUMG/pos-las-casas-del-rincon
Tablero Kanban	https://github.com/users/AndrewUMG/projects/1
Mockup en Figma	https://www.figma.com/design/X00YwF6wiM0zh9zMWQiNqq
Diagramas	https://github.com/AndrewUMG/pos-las-casas-del-rincon/tree/main/docs